

如何解决系统深分页问题

优化项目代码过程中发现一个千万级数据深分页问题，缘由是这样的。

库里有一张耗材 MCS_PROD 表，~~通过同步外部数据中台多维度数据，在系统内部组装为单一耗材产品~~，最终同步到 ES 搜索引擎。

MySQL 同步 ES 流程如下：

1. 通过定时任务的形式触发同步，比如间隔半天或一天的时间频率。
2. 同步的形式为增量同步，根据更新时间的机制，比如第一次同步查询 $\geq 1970-01-01 00:00:00.0$ 。
3. 记录最大的更新时间进行存储，下次更新同步以此为条件。
4. 以分页的形式获取数据，当前页数量加一，循环到最后一页。

在这里问题也就出现了，**MySQL 查询分页 OFFSET 越深入，性能越差**，初步估计线上 MCS_PROD 表中记录在 1000w 左右。

如果按照每页 10 条，OFFSET 值会拖垮查询性能，进而形成一个 "**性能深渊**"。

同步类代码针对此问题有两种优化方式：

1. **采用游标、流式方案进行优化。**
2. **优化深分页性能，文章围绕这个题目展开。**

软硬件说明

MySQL VERSION

```
mysql> select version();
+-----+
| version() |
+-----+
| 5.7.30    |
+-----+
1 row in set (0.01 sec)
```

表结构说明

借鉴公司表结构，字段、长度以及名称均已删减。

```
mysql> DESC MCS_PROD;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| MCS_PROD_ID    | int(11)       | NO   | PRI | NULL    | auto_increment |
| MCS_CODE       | varchar(100)  | YES  |     |         |                |
| MCS_NAME       | varchar(500)  | YES  |     |         |                |
| UPDT_TIME      | datetime      | NO   | MUL | NULL    |                |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

通过测试同学帮忙造了 500w 左右数据量。

```
mysql> SELECT COUNT(*) FROM MCS_PROD;
+-----+
| count(*) |
+-----+
| 5100000 |
+-----+
1 row in set (1.43 sec)
```

因为功能需要满足 **增量拉取的方式**，所以会有数据更新时间的条件查询，以及相关 **查询排序（此处有坑）**。

SQL 语句如下：

```
SELECT
    MCS_PROD_ID,
    MCS_CODE,
    MCS_NAME,
    UPDT_TIME
FROM
    MCS_PROD
WHERE
```

```
UPDT_TIME >= '1970-01-01 00:00:00.0' ORDER BY UPDT_TIME
LIMIT xx, xx
```

重新认识 MySQL 分页

LIMIT 子句可以被用于强制 SELECT 语句返回指定的记录数。LIMIT 接收一个或两个数字参数，参数必须是一个整数常量。

如果给定两个参数，第一个参数指定第一个返回记录行的偏移量，第二个参数指定返回记录行的最大数。

举个简单的例子，分析下 SQL 查询过程，掌握深分页性能为什么差。

```
mysql> SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME FROM MCS_PROD WHERE (UPDT_TIME >= '1970-01-01
00:00:00.0') ORDER BY UPDT_TIME LIMIT 100000, 1;
```

```
+-----+-----+-----+-----+
| MCS_PROD_ID | MCS_CODE          | MCS_NAME          | UPDT_TIME          |
+-----+-----+-----+-----+
|      181789 | XA601709733186213015031 | 尺、桡骨LC-DCP骨板 | 2020-10-19 16:22:19 |
+-----+-----+-----+-----+
1 row in set (3.66 sec)
```

```
mysql> EXPLAIN SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME FROM MCS_PROD WHERE (UPDT_TIME >=
'1970-01-01 00:00:00.0') ORDER BY UPDT_TIME LIMIT 100000, 1;
```

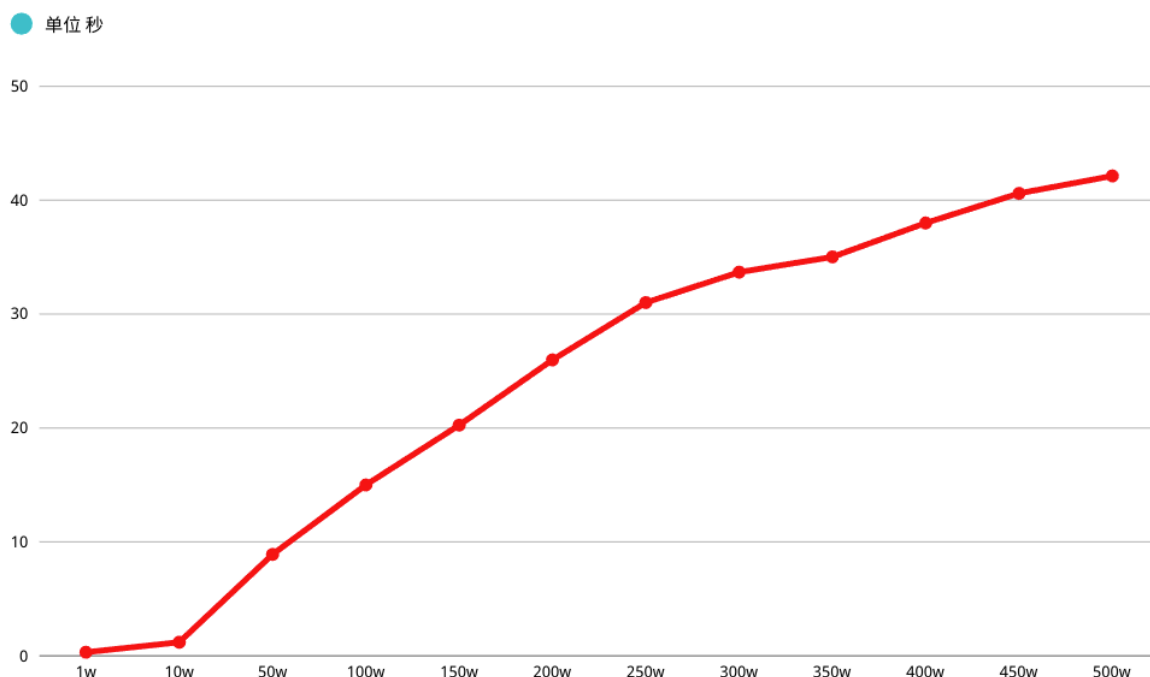
```
+---+-----+-----+-----+-----+-----+-----+-----+
-+-+-----+-----+-----+-----+
| id | select_type | table  | partitions | type  | possible_keys | key          | key_len
| ref | rows       | filtered | Extra      |
+---+-----+-----+-----+-----+-----+-----+-----+
-+-+-----+-----+-----+-----+
| 1  | SIMPLE     | MCS_PROD | NULL       | range | MCS_PROD_1    | MCS_PROD_1  | 5
| NULL | 2296653   | 100.00 | Using index condition |
+---+-----+-----+-----+-----+-----+-----+-----+
-+-+-----+-----+-----+-----+
1 row in set, 1 warning (0.01 sec)
```

简单说明下上面 SQL 执行过程：

1. 首先查询了表 MCS_PROD，进行过滤 UPDT_TIME 条件，查询出展示列（涉及回表操作）进行排序以及 LIMIT。
2. LIMIT 100000, 1 的意思是扫描满足条件的 100001 行，**然后扔掉前 100000 行。**

MySQL 耗费了 **大量随机 I/O 在回表查询聚簇索引的数据上**，而这 100000 次随机 I/O 查询数据不会出现在结果集中。

如果系统并发量稍微高一点，每次查询扫描超过 100000 行，性能肯定堪忧，另外 **LIMIT 分页 OFFSET 越深，性能越差（多次强调）。**



深分页优化

关于 MySQL 深分页优化常见的大概有以下三种策略：

1. 子查询优化。
2. 延迟关联。
3. 书签记录。

上面三点都能大大的提升查询效率，**核心思想就是让 MySQL 尽可能扫描更少的页面**，获取需要访问的记录后再根据关联列回原表查询所需要的列。

1. 子查询优化

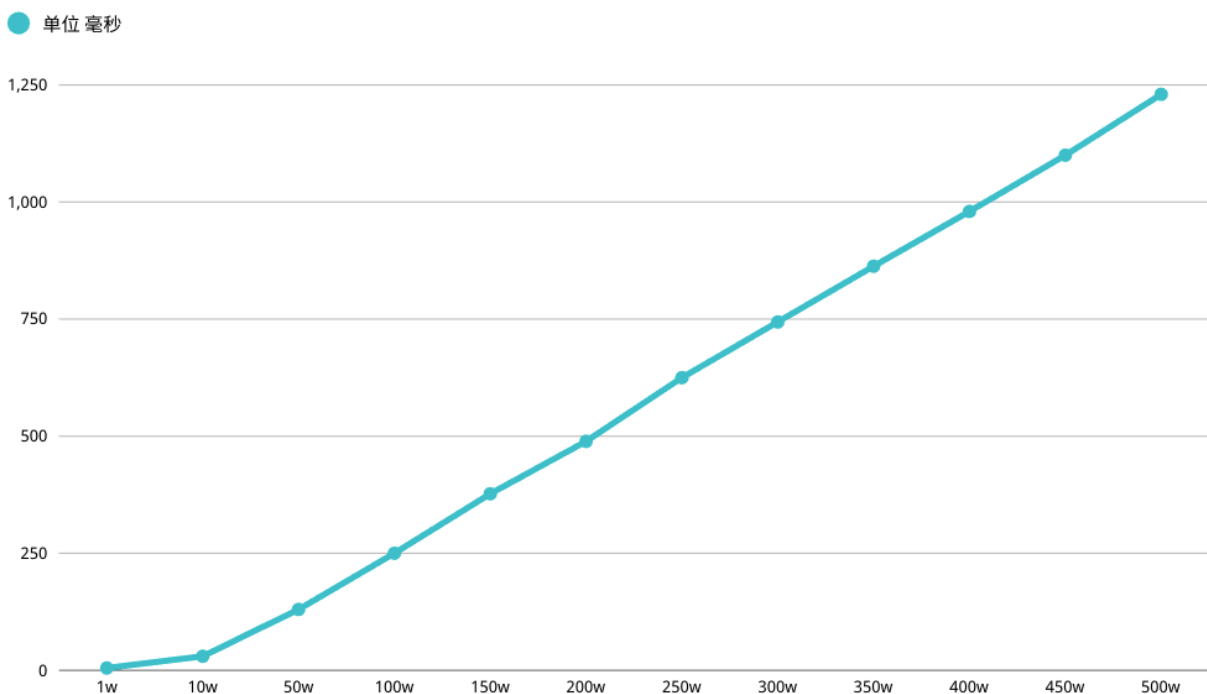
子查询深分页优化语句如下：

```
mysql> SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME FROM MCS_PROD WHERE MCS_PROD_ID >= ( SELECT
m1.MCS_PROD_ID FROM MCS_PROD m1 WHERE m1.UPDT_TIME >= '1970-01-01 00:00:00.0' ORDER BY
m1.UPDT_TIME LIMIT 3000000, 1) LIMIT 1;
+-----+-----+-----+
| MCS_PROD_ID | MCS_CODE          | MCS_NAME          |
+-----+-----+-----+
|      3021401 | XA892010009391491861476 | 金属解剖型接骨板T型接骨板A |
+-----+-----+-----+
1 row in set (0.76 sec)
```



```
mysql> EXPLAIN SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME FROM MCS_PROD WHERE MCS_PROD_ID >= (
SELECT m1.MCS_PROD_ID FROM MCS_PROD m1 WHERE m1.UPDT_TIME >= '1970-01-01 00:00:00.0'
ORDER BY m1.UPDT_TIME LIMIT 3000000, 1) LIMIT 1;
+---+-----+-----+-----+-----+-----+-----+-----+
--+---+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table  | partitions | type  | possible_keys | key          | key_len
| ref | rows       | filtered | Extra              |
+---+-----+-----+-----+-----+-----+-----+-----+
--+---+-----+-----+-----+-----+-----+-----+-----+
| 1  | PRIMARY    | MCS_PROD | NULL         | range | PRIMARY       | PRIMARY      | 4
| NULL | 2296653    | 100.00 | Using where      |
| 2  | SUBQUERY   | m1       | NULL         | range | MCS_PROD_1    | MCS_PROD_1   | 5
| NULL | 2296653    | 100.00 | Using where; Using index |
+---+-----+-----+-----+-----+-----+-----+-----+
--+---+-----+-----+-----+-----+-----+-----+-----+
2 rows in set, 1 warning (0.77 sec)
```

根据执行计划得知，子查询 table m1 查询是用到了索引。首先在 **索引上拿到了聚集索引的主键ID 省去了回表操作**，然后第二查询直接根据第一个查询的 ID 往后再去查 10 个就可以了。



2. 延迟关联

"延迟关联" 深分页优化语句如下:

```
mysql> SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME FROM MCS_PROD INNER JOIN (SELECT
m1.MCS_PROD_ID FROM MCS_PROD m1 WHERE m1.UPDT_TIME >= '1970-01-01 00:00:00.0' ORDER BY
m1.UPDT_TIME LIMIT 3000000, 1) AS MCS_PROD2 USING(MCS_PROD_ID);
```

```
+-----+-----+-----+
| MCS_PROD_ID | MCS_CODE          | MCS_NAME          |
+-----+-----+-----+
|      3021401 | XA892010009391491861476 | 金属解剖型接骨板T型接骨板A |
+-----+-----+-----+
```

1 row in set (0.75 sec)

```
mysql> EXPLAIN SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME FROM MCS_PROD INNER JOIN (SELECT
m1.MCS_PROD_ID FROM MCS_PROD m1 WHERE m1.UPDT_TIME >= '1970-01-01 00:00:00.0' ORDER BY
m1.UPDT_TIME LIMIT 3000000, 1) AS MCS_PROD2 USING(MCS_PROD_ID);
```

```
+---+-----+-----+-----+-----+-----+-----+
----+-----+-----+-----+-----+-----+-----+
| id | select_type | table      | partitions | type  | possible_keys | key      |
key_len | ref          | rows      | filtered  | Extra          |
+---+-----+-----+-----+-----+-----+-----+
----+-----+-----+-----+-----+-----+-----+
```

1	PRIMARY	<derived2>	NULL	ALL	NULL	NULL	NULL
		2296653	100.00	NULL			
1	PRIMARY	MCS_PROD	NULL	eq_ref	PRIMARY	PRIMARY	4
		MCS_PROD2.MCS_PROD_ID	1	100.00	NULL		
2	DERIVED	m1	NULL	range	MCS_PROD_1	MCS_PROD_1	5
		2296653	100.00	Using where; Using index			

3 rows in set, 1 warning (0.00 sec)

思路以及性能与子查询优化一致，只不过采用了 JOIN 的形式执行。

3. 书签记录

关于 LIMIT 深分页问题，核心在于 OFFSET 值，它会 **导致 MySQL 扫描大量不需要的记录行然后抛弃掉**。

我们可以先使用书签 **记录获取上次取数据的位置**，下次就可以直接从该位置开始扫描，这样可以 **避免使用 OFFEST**。

假设需要查询 3000000 行数据后的第 1 条记录，查询可以这么写。

```
mysql> SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME FROM MCS_PROD WHERE MCS_PROD_ID > 3000000
ORDER BY UPDT_TIME LIMIT 1;
```

MCS_PROD_ID	MCS_CODE	MCS_NAME
127	XA683240878449276581799	股骨近端-1螺纹孔锁定板（纯钛）YJBL01

1 row in set (0.00 sec)

```
mysql> EXPLAIN SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME FROM MCS_PROD WHERE MCS_PROD_ID >
3000000 ORDER BY UPDT_TIME LIMIT 1;
```

id	select_type	table	partitions	type	possible_keys	key	key_len
ref	rows	filtered	Extra				
1	SIMPLE	MCS_PROD	NULL	index	PRIMARY	MCS_PROD_1	5

```
+-----+-----+-----+-----+-----+-----+-----+
-+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

不过这种方式局限性也比较大，需要一种类似连续自增的字段，以及业务所能包容的连续概念，视情况而定。



以下言论可能会打破你对 order by 所有美好 YY。

先说结论吧，当 LIMIT OFFSET 过深时，会使 **ORDER BY 普通索引失效**（联合、唯一这些索引没有测试）。

-----+-----+-----+-----+-----+-----+-----+
-+--+


```

| id | select_type | table | partitions | type | possible_keys | key | key_len
| ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | MCS_PROD | NULL | range | MCS_PROD_1 | MCS_PROD_1 | 5
| NULL | 2296653 | 100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

```

先来说一下这个 ORDER BY 执行过程：

1. 初始化 SORT_BUFFER，放入 MCS_PROD_ID,MCS_CODE,MCS_NAME,UPDT_TIME 四个字段。
2. 从索引 UPDT_TIME 找到满足条件的主键 ID，回表查询出四个字段值存入 SORT_BUFFER。
3. 从索引处继续查询满足 UPDT_TIME 条件记录，继续执行步骤 2。
4. 对 SORT_BUFFER 中的数据按照 UPDT_TIME 排序。
5. 排序成功后取出符合 LIMIT 条件的记录返回客户端。

按照 UPDT_TIME 排序可能在内存中完成，也可能需要使用外部排序，取决于排序所需的内存和参数 SORT_BUFFER_SIZE。

SORT_BUFFER_SIZE 是 MySQL 为排序开辟的内存。如果排序数据量小于 SORT_BUFFER_SIZE，排序会在内存中完成。如果数据量过大，内存放不下，**则会利用磁盘临时文件排序。**

针对 SORT_BUFFER_SIZE 这个参数在网上查询到有用资料比较少，大家如果测试过程中存在问题，可以加微信一起沟通。

ORDER BY 索引失效举例

OFFSET 100000 时，通过 key Extra 得知，没有使用磁盘临时文件排序，这个时候把 OFFSET 调整到 500000。

一首凉凉送给写这个 SQL 的同学，发现了 Using filesort。

```
mysql> EXPLAIN SELECT MCS_PROD_ID,MCS_CODE,MCS_NAME,UPDT_TIME FROM MCS_PROD WHERE
(UPDT_TIME >= '1970-01-01 00:00:00.0') ORDER BY UPDT_TIME LIMIT 500000, 1;
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref |
| rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | MCS_PROD | NULL | ALL | MCS_PROD_1 | NULL | NULL | NULL |
| 4593306 | 50.00 | Using where; Using filesort |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

Using filesort 表示在索引之外，**需要额外进行外部的排序动作**，性能必将受到严重影响。

所以我们应该 **结合相对应的业务逻辑避免常规 LIMIT OFFSET**，采用 **# 深分页优化** 章节进行修改对应业务。

结语

最后有一点需要声明下，MySQL 本身并不适合单表大数据量业务。

因为 MySQL 应用在企业级项目时，针对库表查询并非简单的条件，可能会有更复杂的联合查询，亦或者是大数据量时存在频繁新增或更新操作，维护索引或者数据 ACID 特性上必然存在性能牺牲。

如果设计初期能够预料到库表的数据增长，理应构思合理的重构优化方式，比如 ES 配合查询、分库分表、TiDB 等解决方式。

参考资料：

1. 《高性能 MySQL 第三版》
2. 《MySQL 实战 45 讲》

原文: <<https://www.yuque.com/magestack/12306/loygqy6veku6452w>>